

SAGIP

Demo Script and MVVM Architecture Guide

1. Starting the Demonstration

Run the Flutter application in Chrome:

```
flutter run -d chrome --web-port 3000
```

Speaking cue:

“SAGIP is a campus emergency response and incident management platform for USJ-R. It allows students to register, sign in, view their dashboard, and submit emergency alerts to the Safety and Security Department.”

2. Features Accomplished So Far

1. Supabase database configuration and Flutter connection
2. Student and SSD authentication interface
3. Student registration
4. Forgot-password and reset-password flow
5. Session management and role-based routing
6. Student dashboard
7. Functional SOS reporting

3. Suggested Demo Flow

A. Student Registration

Show the following:

- Student ID only accepts digits.
- Last name, first name, and optional middle initial are separate.
- Birthday is selected through a calendar.
- Department selection filters the available courses.
- Contact number only accepts digits.
- Student email must end in `@usjr.edu.ph`.
- Password and confirmation are validated.

Speaking cue:

“The registration form validates student information before creating an account. Department and course use dependent dropdowns, so only courses belonging to the selected department are displayed.”

B. Login and Session Management

Sign in using a registered student account.

Speaking cue:

“After login, Supabase creates a session. The application reads the authenticated user and their role. A student is routed to the Student Dashboard, while an SSD administrator is routed to the SSD interface.”

C. Student Dashboard

Show:

- Welcome header containing the student's name
- Announcements card
- Quick Actions card
- Incident History shortcut
- Bottom navigation for Home, History, SOS, and Profile
- Sign-out option

Speaking cue:

“The dashboard follows the project wireframe. Some destinations remain placeholders because they belong to separate Kanban tasks, but the navigation foundation is already functional.”

D. SOS Reporting

Open SOS from the Quick Actions card or the central SOS button.

- Select an emergency type loaded from Supabase.
- Enter optional incident details.
- Press SEND SOS.
- Show the confirmation dialog.
- Confirm submission.
- Show the success message.

Speaking cue:

“The selected emergency type, optional details, student identity, pending status, priority, and

submission timestamp are saved as an incident in Supabase. GPS coordinates will be connected under the separate GPS module.”

4. Simple MVVM Explanation

Main explanation:

“We used MVVM to separate the interface, application state, and data. Views display screens, ViewModels handle screen logic and state, Models represent data, and Services communicate with Supabase.”

Model

Models describe the data used by the application.

- lib/features/auth/models/app_user.dart
- lib/features/auth/models/student_registration.dart
- lib/features/sos/models/emergency_type.dart
- lib/features/sos/models/incident_submission.dart

```
class EmergencyType {  
  final String id;  
  final String name;  
}
```

Speaking cue:

“EmergencyType is a Model. It represents an emergency category retrieved from the database.”

View

Views contain the visible Flutter widgets and collect user input.

- lib/features/auth/views/registration_view.dart
- lib/features/dashboard/views/student_dashboard_view.dart
- lib/features/sos/views/sos_view.dart

Speaking cue:

“The Views contain forms, buttons, dropdowns, cards, and navigation. They do not directly query or insert records into Supabase.”

ViewModel

ViewModels store screen state and handle actions requested by the View.

- lib/features/auth/viewmodels/registration_view_model.dart
- lib/features/auth/viewmodels/session_view_model.dart

- lib/features/dashboard/viewmodels/student_dashboard_view_model.dart
- lib/features/sos/viewmodels/sos_view_model.dart

```
void selectTab(StudentDashboardTab tab) {
  _selectedTab = tab;
  notifyListeners();
}
```

Speaking cue:

“The Student Dashboard ViewModel stores the selected tab. When the value changes, notifyListeners tells the View to redraw.”

The SOS ViewModel contains:

```
Future<bool> submit(String details) async
```

Speaking cue:

“The SOS ViewModel validates the selected emergency category, manages loading and error states, creates the incident submission, and asks the service to save it.”

Service / Data Layer

Services are responsible for communication with Supabase.

- lib/features/auth/services/supabase_auth_service.dart
- lib/features/sos/services/supabase_incident_service.dart

```
Future<void> submitIncident(
  IncidentSubmission submission,
) async
```

Speaking cue:

“Only the service layer accesses Supabase. The incident service finds the student record and inserts the emergency incident into the database.”

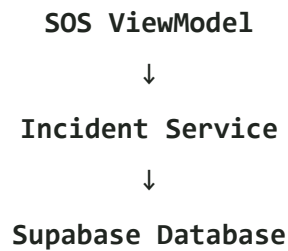
5. MVVM Flow During SOS Submission

Student presses SEND SOS



SOS View





6. Session Management

Main files:

- `lib/features/auth/viewmodels/session_view_model.dart`
- `lib/features/auth/views/session_gate.dart`

Speaking cue:

“Session management remembers whether a user is logged in. SessionGate checks the authenticated user’s role. Students see the Student Dashboard, administrators see the SSD interface, and signed-out users see Login.”

7. Why MVVM Was Used

Speaking cue:

“MVVM keeps the user interface separate from screen logic and database operations. This makes SAGIP easier to maintain, test, and expand without placing all responsibilities in one file.”

8. If Asked About Current Limitations

- GPS coordinates are not yet captured because GPS is a separate task.
- Incident History and Student Profile currently have placeholders.
- SSD Dashboard and notification modules will be developed separately.
- Email confirmation may be disabled temporarily during development.

Speaking cue:

“The current build demonstrates the working foundation. Remaining modules are separated into Kanban tasks and can be integrated without changing the existing MVVM architecture.”